



# Data Converters

## Chapter 07 – Analog-to-Digital Converters

Summer Term 2026  
Dr.-Ing. Matthias Keller

UNI  
FREIBURG

Laboratory for Microelectronics  
Department of Microsystems Engineering – IMTEK  
University of Freiburg



## Acknowledgement

Concepts & slides are partially based on the lecture

### VLSI Data Conversion Circuits

held by

**Prof. B. Murmann, PhD, University of Hawai'i**

during his sabbatical at the University of Freiburg in 2009



UNI  
FREIBURG



## Overview

- Flash ADC
- Two-Step Flash & Pipeline ADC
- Integrating ADCs
- Bit-at-a-Time ADCs

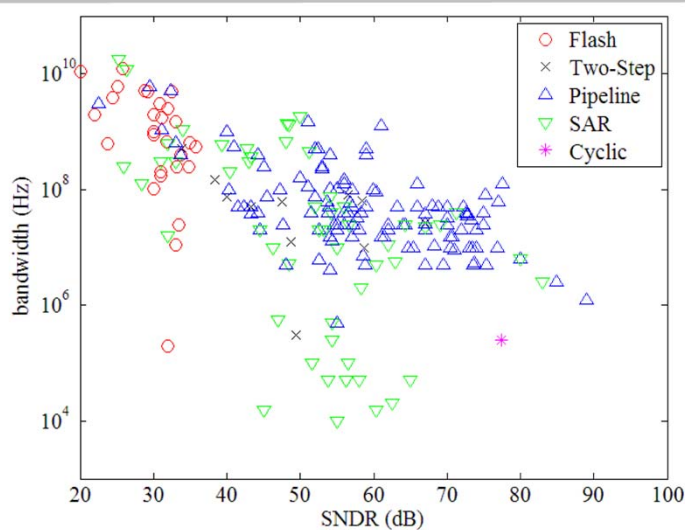
26-06-25

Data Converters

3

UNI  
FREIBURG

## State Of The Art – Overview



based on the data provided at  
B. Murmann, "ADC Performance Survey 1997-2013," [Online]  
available: <http://www.stanford.edu/~murmman/adcsurvey.html>

26-06-25

Data Converters

4

UNI  
FREIBURG



## Overview

- **Flash ADC**
  - Interpolation
  - Folding
- Two-Step Flash & Pipeline ADC
- Integrating ADCs
- Bit-at-a-Time ADCs

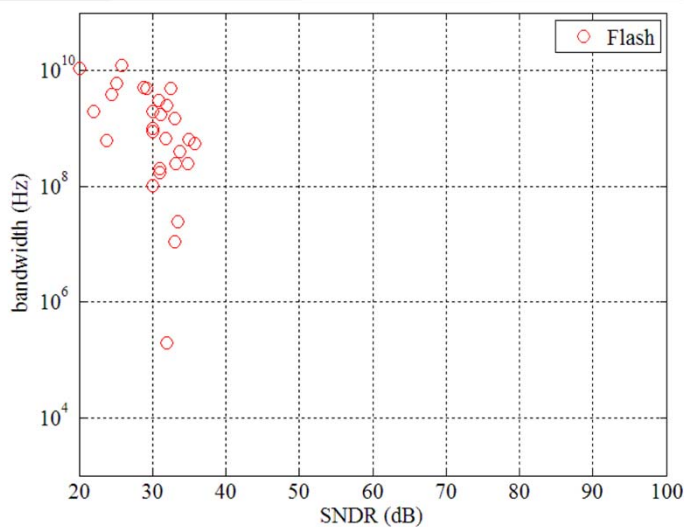
26-06-25

Data Converters

5

UNI  
FREIBURG

## Flash ADC – Overview State Of The Art



based on the data provided at  
B. Murmann, "ADC Performance Survey 1997-2013," [Online]  
available: <http://www.stanford.edu/~murmman/adcsurvey.html>

26-06-25

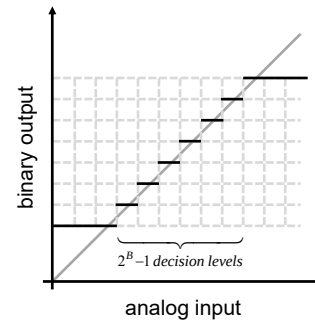
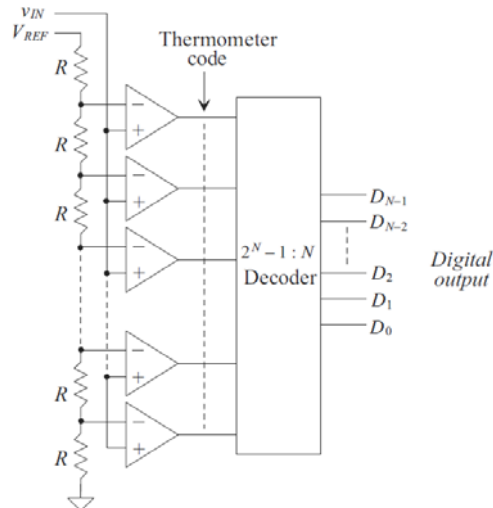
Data Converters

6

UNI  
FREIBURG



## Flash ADC



- conversion time:  $T_s = 1/f_s$
- 6-bit Flash ADC:  
64 resistors =  $2^B$   
63 comparators =  $2^B - 1$

26-06-25

Data Converters

7

UNI  
FREIBURG

## Flash ADC – Resistor Mismatch & DNL

$$DNL(k) = \frac{W(k) - W_{\text{avg}}}{W_{\text{avg}}} = \frac{R_k - R}{R} = \frac{\Delta R}{R}$$

$$\text{stdev}[DNL(k)] = \text{stdev}\left(\frac{\Delta R}{R}\right) = \sigma_R$$

The standard deviation of  $DNL$  for each code equals the relative matching  $\sigma_R$  of the resistors.

see chapter "DAC"

26-06-25

Data Converters

8

UNI  
FREIBURG



## Flash ADC – Resistor Mismatch & INL

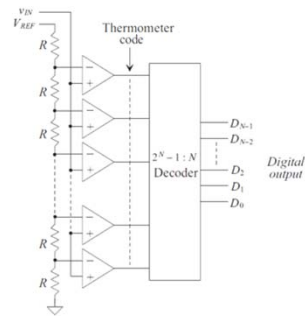
$$INL(k) = \frac{T(k) - T_{\text{uniform}}(k)}{W_{\text{avg}}}$$

$$= \frac{\sum_{j=1}^k R_j \frac{V_{\text{ref}}}{\sum_{j=1}^{2^B} R_j} - k \frac{\sum_{j=1}^{2^B} R_j}{N} \frac{V_{\text{ref}}}{\sum_{j=1}^{2^B} R_j}}{\frac{\sum_{j=1}^{2^B} R_j}{N} \frac{V_{\text{ref}}}{\sum_{j=1}^{2^B} R_j}}$$

“average” resistor      current

$$= \frac{N \sum_{j=1}^k R_j}{\sum_{j=1}^k R_j + \sum_{j=k+1}^{2^B} R_j} - k = \frac{N \cdot A}{A + B} - k$$

$$\text{var}[INL(k)] \approx k \left(1 - \frac{k}{N}\right) \sigma_R^2 \rightarrow \sigma_{INL}(k) \approx \sqrt{k \left(1 - \frac{k}{N}\right)} \sigma_R$$



see chapter “DAC”

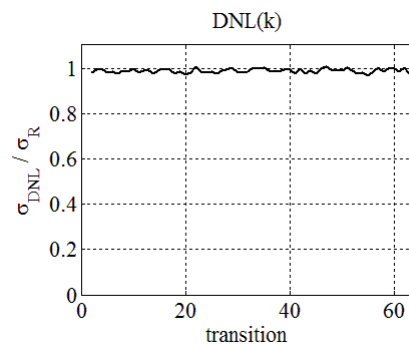
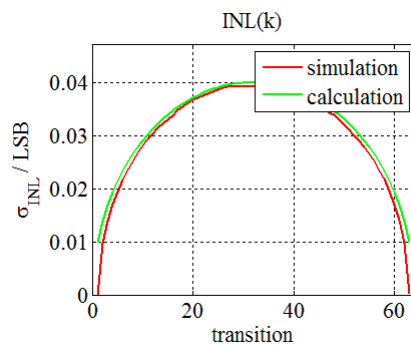
26-06-25

Data Converters

9

UNI  
FREIBURG

## Flash ADC – Simulation INL



6-bit Flash ADC, i.e.,  $B = 6$   
statistics of 5000 transfer characteristics ( $\sigma_R = 1\%$ )

26-06-25

Data Converters

10

UNI  
FREIBURG



## Overview

- **Flash ADC**
  - **Interpolation**
  - Folding
- Two-Step Flash & Pipeline ADC
- Integrating ADCs
- Bit-at-a-Time ADCs

26-06-25

Data Converters

11

UNI  
FREIBURG

## Flash ADC – Reducing Complexity

Example: 10-bit Flash ADC

Compared to the previous 6-bit example

- 16x the numbers of comparators are used
- size and power of each comparator are to be increased by  $16^2$  due to matching
- input capacitance increases to  $1 \text{ pF} \cdot 16^3 = 4.096 \text{ nF}$
- power consumption increases to  $500 \text{ mW} \cdot 16^3 = 2048 \text{ W}$

Techniques for overcoming these limitations

- interpolation
- folding
- multi-step conversion & pipelining

26-06-25

Data Converters

12

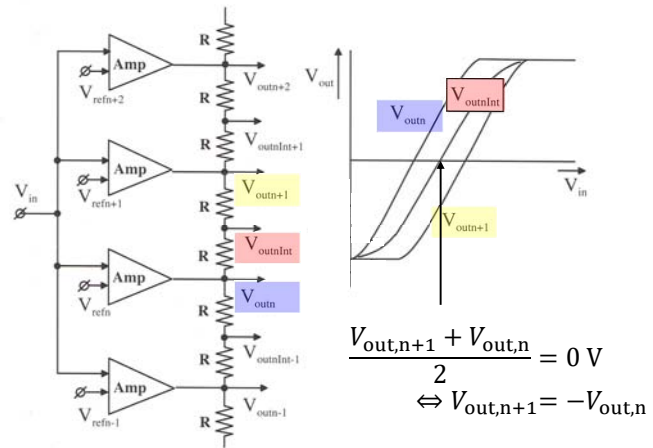
UNI  
FREIBURG



## Interpolation

Idea: Interpolation between the outputs of the preamplifiers

- reduces number of preamplifiers  
(less capacitance, less area, less power dissipation)
- same number of latches
- nice „side-benefit“: decreased sensitivity to offset of the preamplifiers  
→ improved DNL



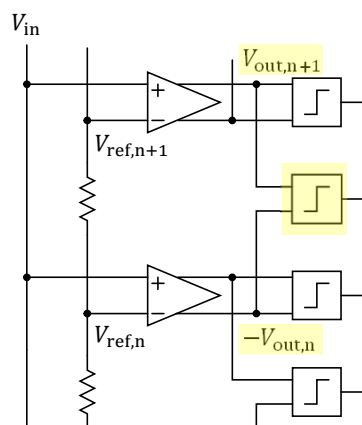
26-06-25

Data Converters

13

UNI  
FREIBURG

## Interpolation – Differential Implementation



$$\frac{V_{out,n+1} + V_{out,n}}{2} = 0 \text{ V}$$

$$\Leftrightarrow V_{out,n+1} = -V_{out,n}$$

26-06-25

Data Converters

14

UNI  
FREIBURG

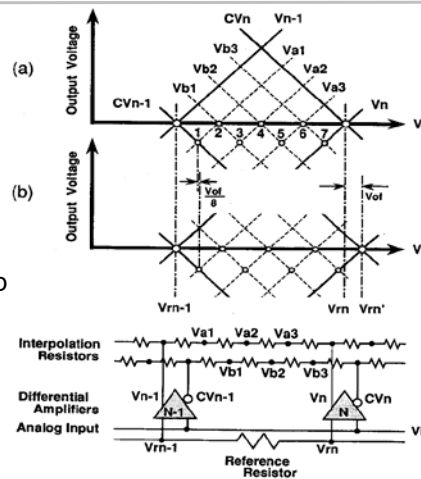


## Higher Order Interpolation

- higher order interpolation = more levels
- interpolation factor  $M = \frac{\# \text{ latches}}{\# \text{ preamplifiers}}$   
this example:  $M = 8$

### limiting factors:

- linear range of adjacent preamplifiers must overlap  
→ upper bound on gain of preamplifiers
- resistor string reduces signal bandwidth  
→  $M$  typically around 4
- preamplifiers' nonlinearity may limit the precision  
of zero crossings for  $M > 2$



Kimura et al., "A 10-b 300-MHz Interpolated-Parallel A/D Converter," IEEE J. Solid-State Circuits, pp. 438-446, vol. 28, no. 4, April 1993

26-06-25

Data Converters

15

UNI  
FREIBURG

## Overview

- **Flash ADC**
  - Interpolation
  - **Folding**
- Two-Step Flash & Pipeline ADC
- Integrating ADCs
- Bit-at-a-Time ADCs

26-06-25

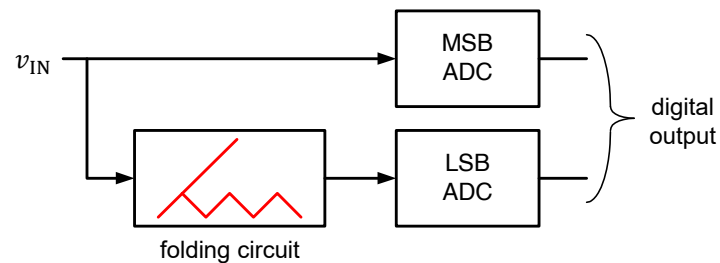
Data Converters

16

UNI  
FREIBURG



## Folding ADC



- fast
- significantly fewer comparators than flash ADC due to reuse of LSB ADC
  - nonidealities of folding circuit limit resolution to approx. 10 bits
  - invented in the late 70's / early 80's

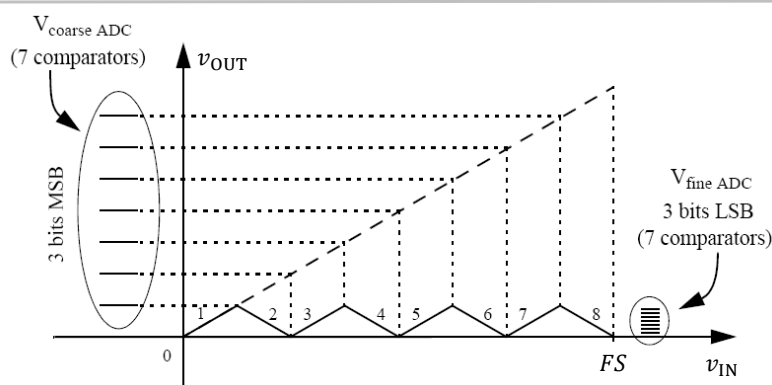
26-06-25

Data Converters

17

UNI  
FREIBURG

## Example – 6-bit Folding ADC



- coarse ADC determines segment
- fine ADC determines level within segment
- folding factor  $F_F$  determines number of folded segments

6-bit flash ADC  
→ 64 comparators

6-bit folding ADC  
→ 14 comparators

26-06-25

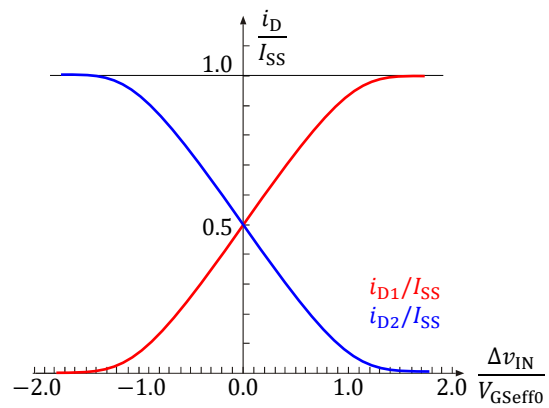
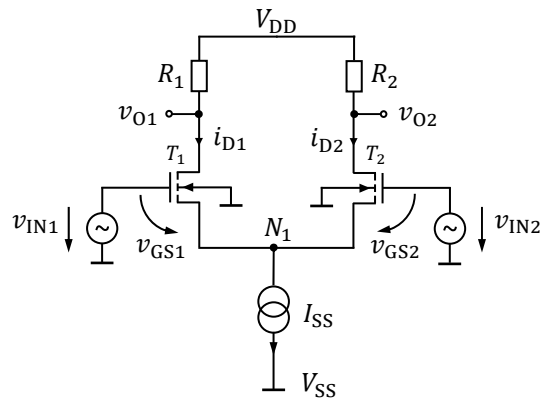
Data Converters

18

UNI  
FREIBURG



## Implementation Folding I



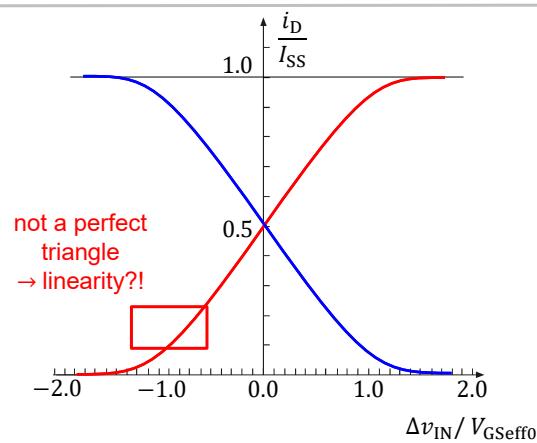
26-06-25

Data Converters

19

UNI  
FREIBURG

## Implementation Folding II



Use several differential stages with different bias voltages!

26-06-25

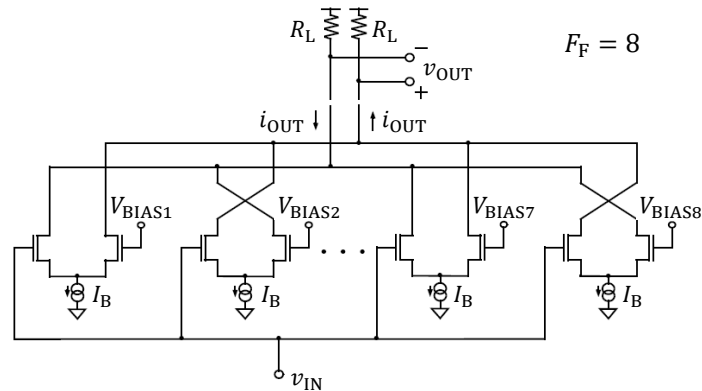
Data Converters

20

UNI  
FREIBURG



## Implementation Folding III



- $V_{BIAS1} < V_{BIAS2} < \dots < V_{BIAS8}$
- only one differential pair is active for any  $v_{IN}$  (all others are saturated)

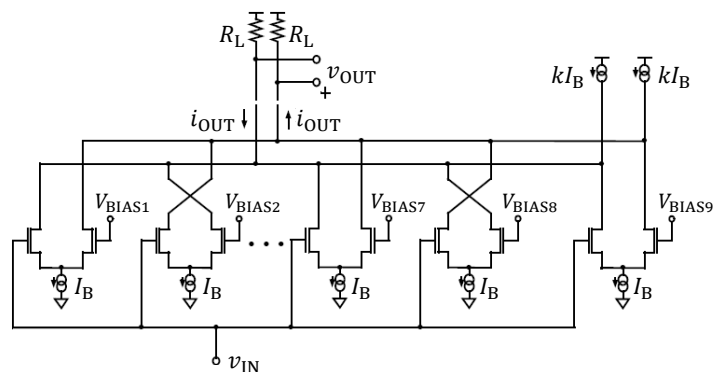
26-06-25

Data Converters

21

UNI  
FREIBURG

## Folding – Improved Implementation



- remove DC component of  $v_{OUT}$  due to mismatch by another differential pair
- parameter  $k$  controls output bias point / common-mode voltage

26-06-25

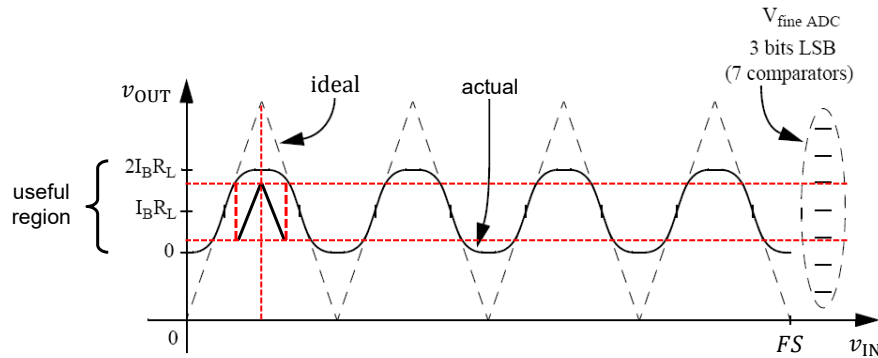
Data Converters

22

UNI  
FREIBURG



## Practice – Rounding Problem



It doesn't make sense to use a 3-bit fine ADC due to the limited linear range!

Reduce resolution of fine ADC & add further folding segments operating in the "linear" range

→ 1x folder with  $F_F = 8$  and 1x 3-bit fine ADC  
 ⇔ 4x folders with  $F_F = 4$  and 1x 2-bit fine ADC (per folder → total 4x 2-bit fine ADCs!)

26-06-25

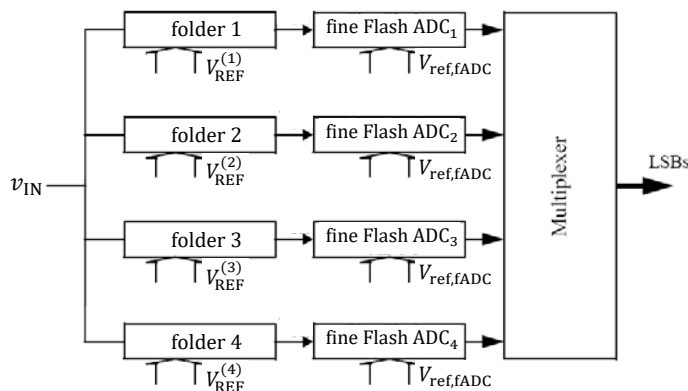
Data Converters

23

UNI  
FREIBURG

## Multiple Folds I

Use several folders so that any input voltage falls into a "linear" region!



4x folders with  $F_F = 4$  and  
 1x 2-bit fine ADC per folder

26-06-25

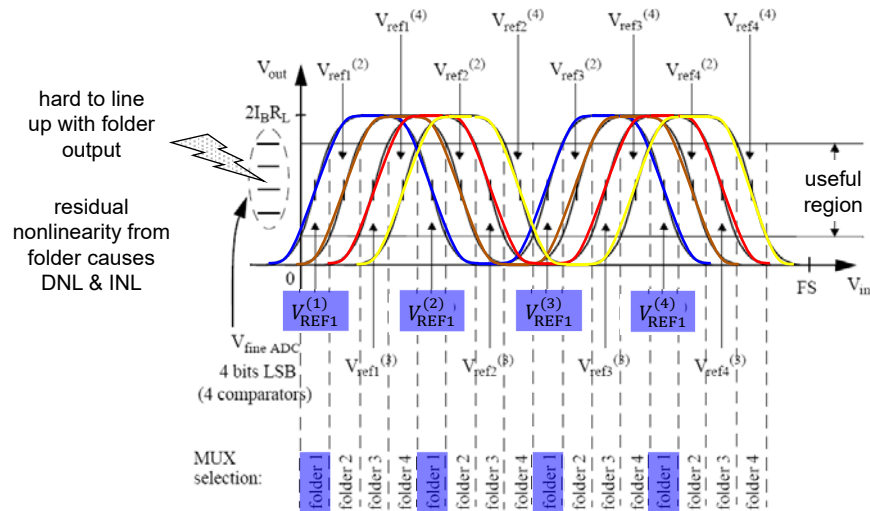
Data Converters

24

UNI  
FREIBURG



## Multiple Folders II



26-06-25

## Data Converters

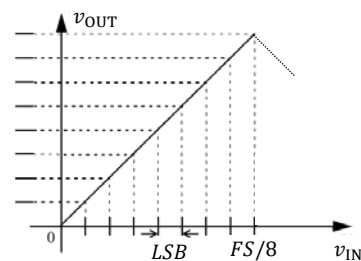
25

UNI  
FREIBURG

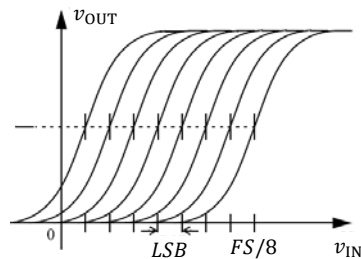


## Example – 6-bit Folding ADC

- 1x folder with  $F_F = 8$  and 1x 3-bit fine ADC
- 7x comparators



8x folders with  $F_F = 8$  and  
1-bit fine ADC per folder  
→ 8x comparators



advantage:  
1-bit ADC does not detect any nonlinearity

26-06-25

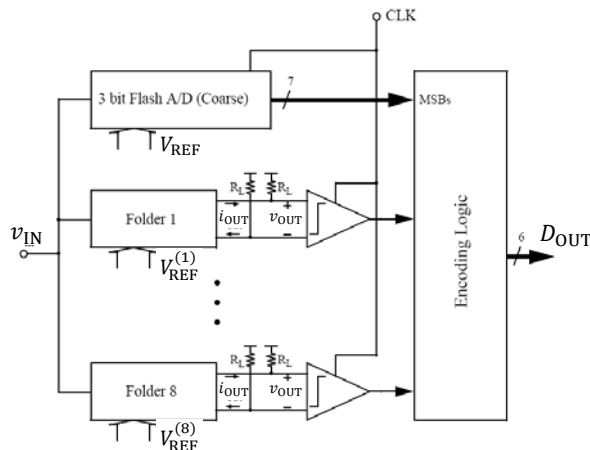
## Data Converters

26


 UNI  
FREIBURG



## Example – 6-bit Folding ADC, 8x Folds, 1-bit ADC



initial idea:

3-bit (coarse) Flash ADC  
 1x folder ( $F_F = 8$ ) & 3-bit (fine) Flash ADC  
 → 14 comparators

currently:

3-bit (coarse) Flash ADC  
 8x folders ( $F_F = 8$ ) & 1-bit (fine) Flash ADC  
 → 15 comparators

How about interpolation?

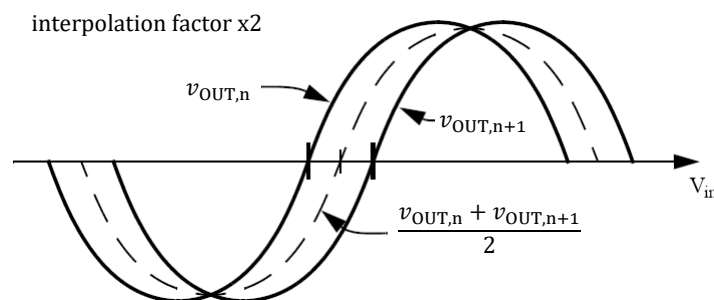
26-06-25

Data Converters

27

UNI  
FREIBURG

## Interpolation



Same idea as discussed in the context of interpolating flash ADCs!

26-06-25

Data Converters

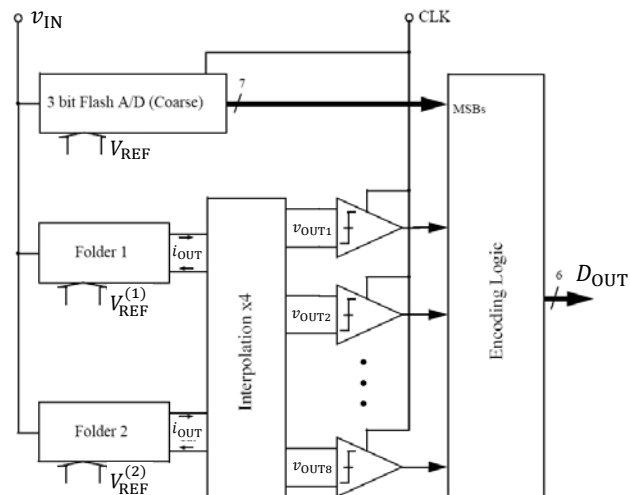
28

UNI  
FREIBURG



## Example – 6-bit Folding & Interpolating ADC

3-bit (coarse) Flash ADC  
 2x folders ( $F_F = 8$ )  
 4x interpolation  
 → save 2x folders ( $F_F = 8$ )  
 & thus 16 differential stages  
 → still 15 comparators



26-06-25

Data Converters

29

UNI  
FREIBURG

## Overview

- Flash ADC
- **Two-Step Flash & Pipeline ADC**
- Integrating ADCs
- Bit-at-a-Time ADCs

26-06-25

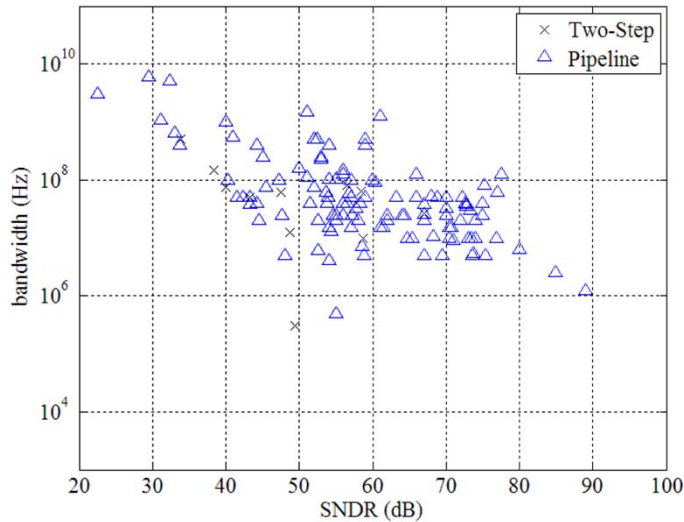
Data Converters

30

UNI  
FREIBURG



## Two-Step & Pipeline – Overview State of the Art



based on the data provided at  
B. Murmann, "ADC Performance Survey 1997-2013," [Online]  
available: <http://www.stanford.edu/~murmann/adcsurvey.html>

26-06-25

Data Converters

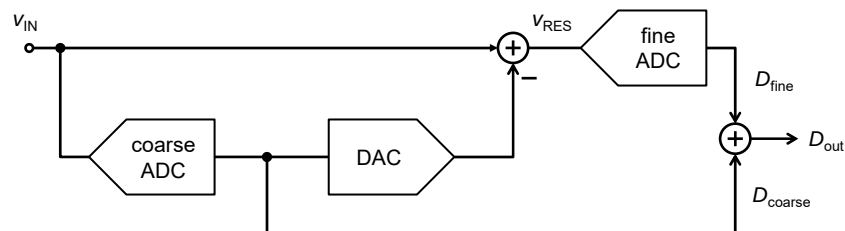
31

UNI  
FREIBURG

## Multi-Step Conversion – Approach

general idea

- perform a “coarse” quantization of the input
- compute residue (error) of first conversion using a DAC and a subtractor
- convert residue using a second “fine” quantizer and digitally add to overall output



26-06-25

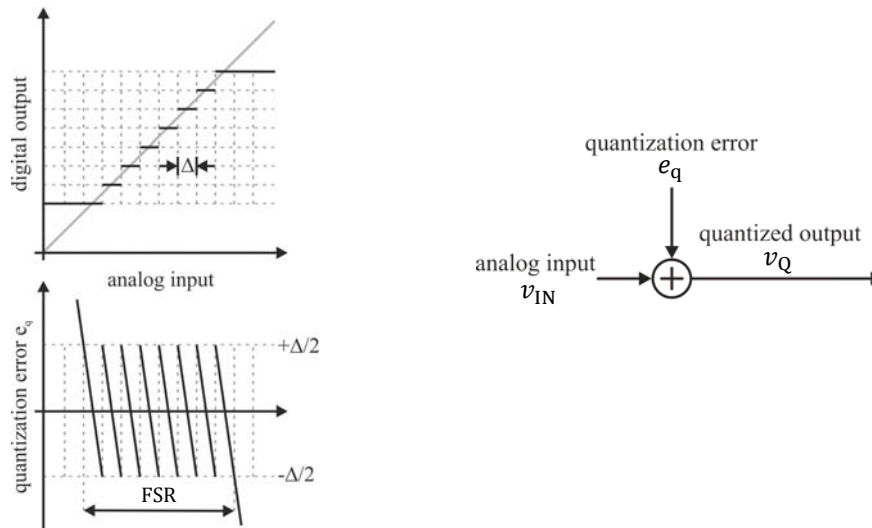
Data Converters

32

UNI  
FREIBURG



## Remember – Linear Quantizer Model



26-06-25

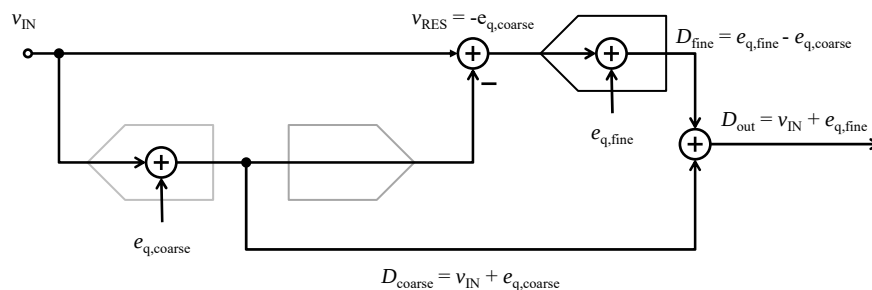
Data Converters

33

UNI  
FREIBURG

## Multi-Step Conversion – Linear Analysis

- ideal DAC  $\rightarrow$  just converts the digital code back to its analog value



- output contains only quantization error of fine DAC

26-06-25

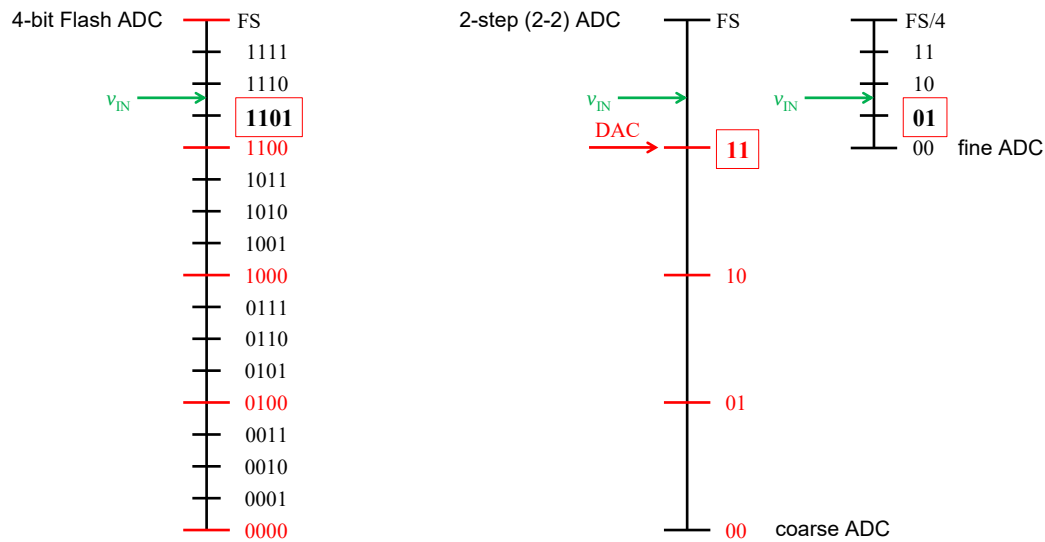
Data Converters

34

UNI  
FREIBURG



## Example – 4-bit Flash vs. 2-step 2-2 ADC



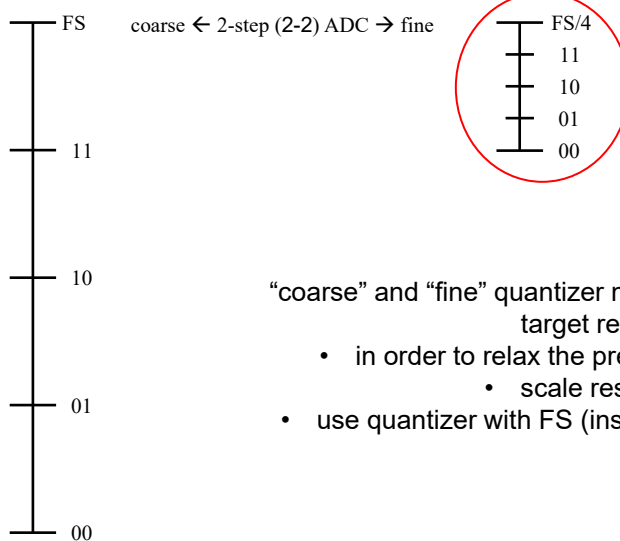
26-06-25

Data Converters

35

UNI  
FREIBURG

## Example – Two-Step ADC Precision



"coarse" and "fine" quantizer must have precision of overall target resolution

- in order to relax the precision of "fine" quantizer
  - scale residue to FS
- use quantizer with FS (instead of FS/4) for conversion

26-06-25

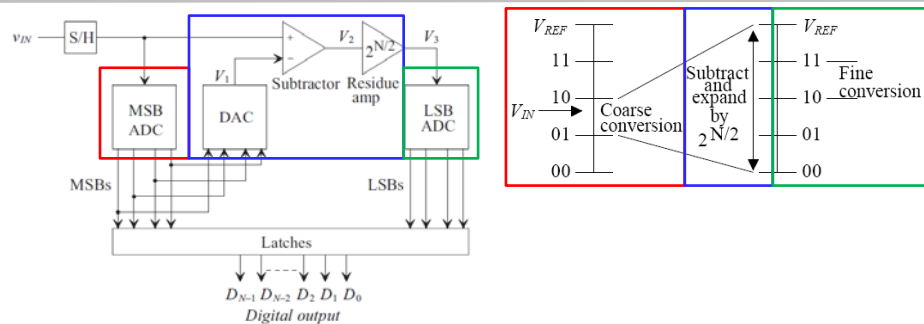
Data Converters

36

UNI  
FREIBURG



## Two-Step Flash ADC



- latency:  $2T_s = 2/f_s$  (first conversion)
  - conversion speed:  $f_s$
- 6-bit Flash ADC  $\rightarrow$  2x 3-bit Flash ADC
  - 2x 8 resistors =  $2 \times 2^{B/2}$
  - 2x 7 comparators =  $2 \times (2^{B/2}-1)$

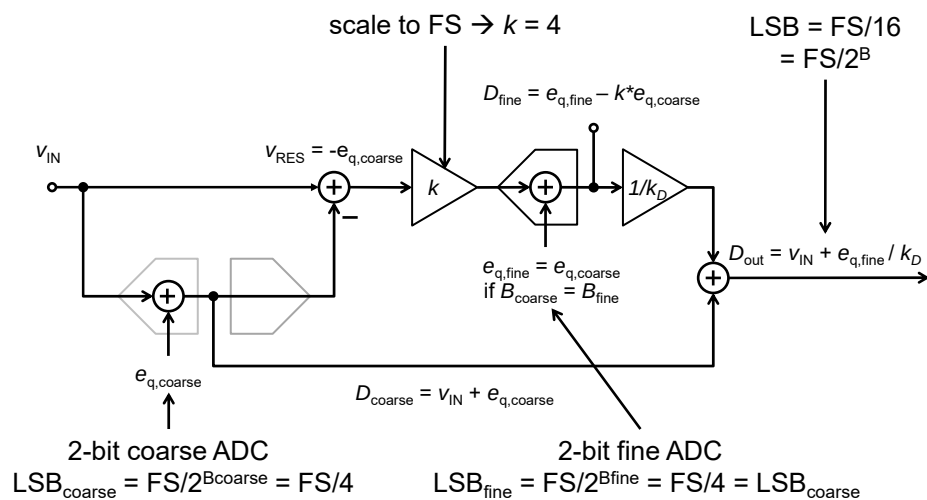
26-06-25

Data Converters

37

UNI  
FREIBURG

## Combining the Outputs – Two-Step 2-2 ADC



26-06-25

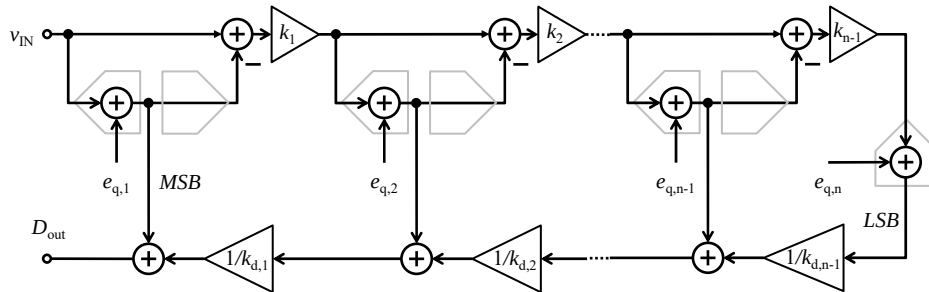
Data Converters

38

UNI  
FREIBURG



## Pipeline ADC – More than Two Stages...



$$D_{out} = v_{IN} + \left(1 - \frac{k_1}{k_{d,1}}\right) e_{q,1} + \frac{1}{k_{d,1}} \left(1 - \frac{k_2}{k_{d,2}}\right) e_{q,2} + \dots + \frac{1}{\prod_{i=1}^{n-2} k_{d,i}} \left(1 - \frac{k_{n-1}}{k_{d,n-1}}\right) e_{q,n-1} + \frac{1}{\prod_{i=1}^{n-1} k_{d,i}} e_{q,n}$$

ideal case, i.e.,  $k_i = k_{d,i}$

$$D_{out} = v_{IN} + \frac{1}{\prod_{i=1}^{n-1} k_{d,i}} e_{q,n}$$

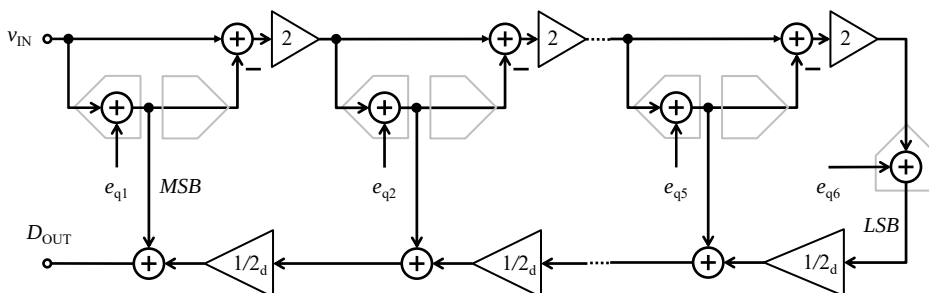
26-06-25

Data Converters

39

UNI  
FREIBURG

## Example – 6-bit Pipeline ADC, 1-Bit per Stage



$$D_{OUT} = v_{IN} + \frac{1}{\prod_{i=1}^5 2_d} e_{q,6} = v_{IN} + \left( -\frac{1}{2^5} \frac{FSR}{2} / 2 \dots + \frac{1}{2^5} \frac{FSR}{2} / 2 \right)$$

- latency:  $6T_s = 6/f_s$  (first conversion)
  - conversion speed:  $f_s$
  - 12 resistors & 6 comparators
- analog summing & multiplication operations?

26-06-25

Data Converters

40

UNI  
FREIBURG



## Overview

- Flash ADC
- Two-Step Flash & Pipeline ADC
- **Integrating ADCs**
  - **Single-Slope ADC**
  - **Dual-Slope ADC**
- Bit-at-a-Time ADCs

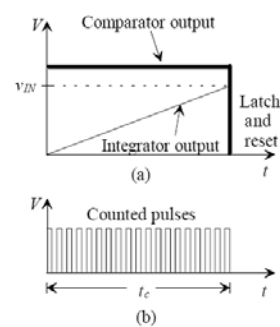
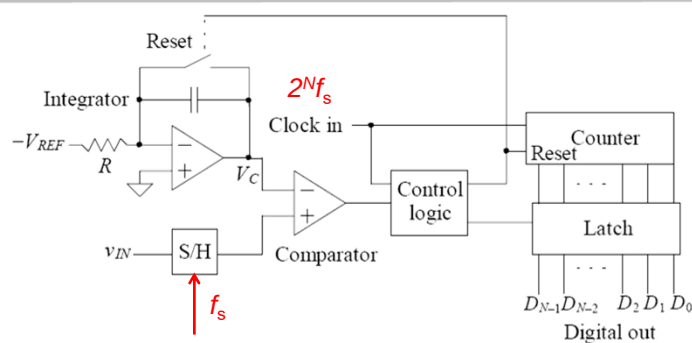
26-06-25

Data Converters

41

UNI  
FREIBURG

## Single-Slope ADC



- conversion speed: depends on input signal
- for low-bandwidth signals
- high resolution  $\leftrightarrow$  low speed

$$t_c = \frac{v_{IN}}{V_{REF}} 2^N T_{CLK}$$

26-06-25

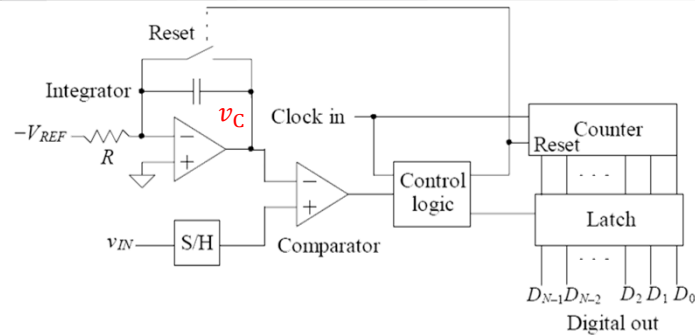
Data Converters

42

UNI  
FREIBURG



## Single-Slope ADC – Nonidealities & Limitation



- voltage  $v_c$

$$v_c = \frac{1}{C} \int_0^{t_c} \frac{V_{REF}}{R} dt = \frac{V_{REF} t_c}{RC} \text{ with } t_c = \frac{v_{IN}}{V_{REF}} 2^N T_{CLK} \rightarrow v_c = \frac{2^N v_{IN}}{f_{CLK} RC}$$

- precise clock, resistor & capacitor required
- offset of comparator  $\rightarrow$  additional / fewer clock cycles

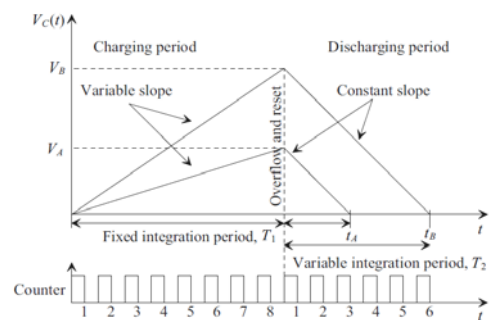
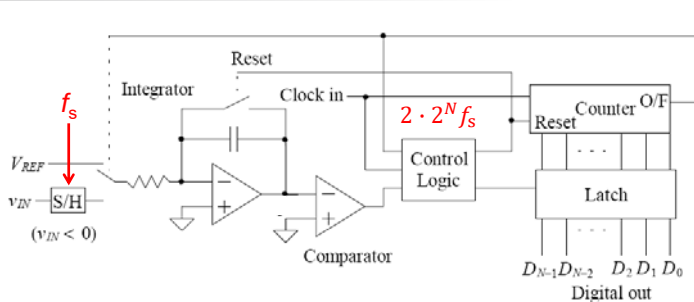
26-06-25

Data Converters

43

UNI  
FREIBURG

## Dual-Slope ADC



integrate  $v_{IN}$  for  $2^B$  clock cycles  $\rightarrow$  variable slope:

$$T_1 = 2^N T_{CLK}$$

de-integrate with  $V_{REF}$  & count # pulses  $\rightarrow$  constant slope:

$$T_2 = \frac{v_{IN}}{V_{REF}} 2^N T_{CLK}$$

26-06-25

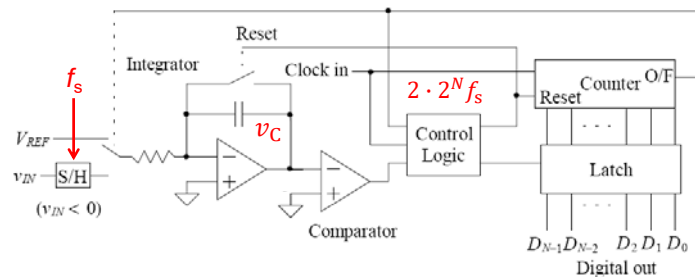
Data Converters

44

UNI  
FREIBURG



## Comparison with Single-Slope ADC



• voltage  $v_C$

$$v_C|_{t=T_1} = \frac{1}{C} \int_0^{T_1} \frac{v_{IN}}{R} dt = \frac{T_1}{RC} v_{IN}$$

$$v_C|_{t=T_2} = \frac{T_1}{RC} v_{IN} - \frac{1}{C} \int_0^{T_2} \frac{V_{REF}}{R} dt = \frac{v_{IN}T_1 - V_{REF}T_2}{RC} = 0 \text{ V} \rightarrow \frac{D_{OUT}}{2^N} = \frac{v_{IN}}{V_{REF}}$$

- result does not depend on precision of resistor, capacitor & clock

26-06-25

Data Converters

45

UNI  
FREIBURG

## Overview

- Flash ADC
- Two-Step Flash & Pipeline ADC
- Integrating ADCs
- **Bit-at-a-Time ADCs**
  - **Cyclic ADC**
  - **Successive Approximation (SAR) ADC**

26-06-25

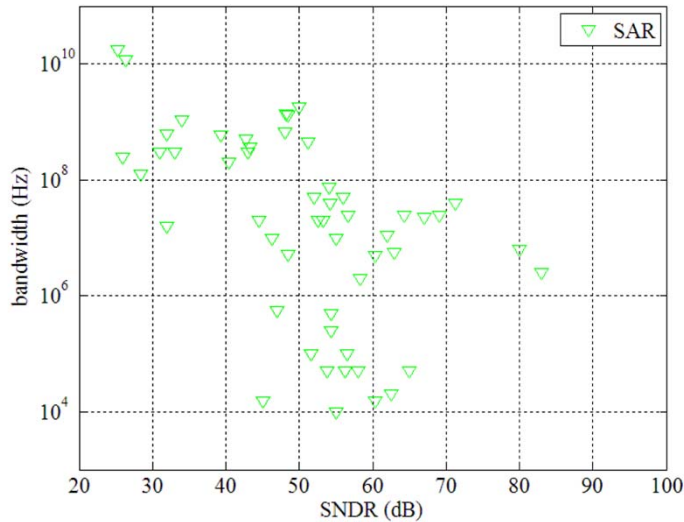
Data Converters

46

UNI  
FREIBURG



## SAR ADC – Overview State of the Art



based on the data provided at  
B. Murmann, "ADC Performance Survey 1997-2013," [Online]  
available: <http://www.stanford.edu/~murmann/adcsurvey.html>

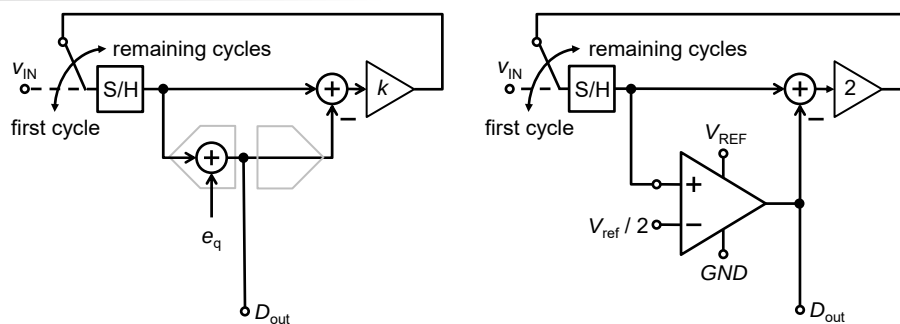
26-06-25

Data Converters

47

UNI  
FREIBURG

## Cyclic ADC



- essentially a single pipeline stage used in a cyclic fashion
- latency:  $B \times T_s = B/f_s$  (first conversion)
- conversion speed:  $B \times T_s$ , i.e., every conversion requires  $B \times f_s$  clock cycles!
- 2 resistors & 1 comparator (independent of  $B$ )
- trade-off: complexity  $\leftrightarrow$  conversion speed

26-06-25

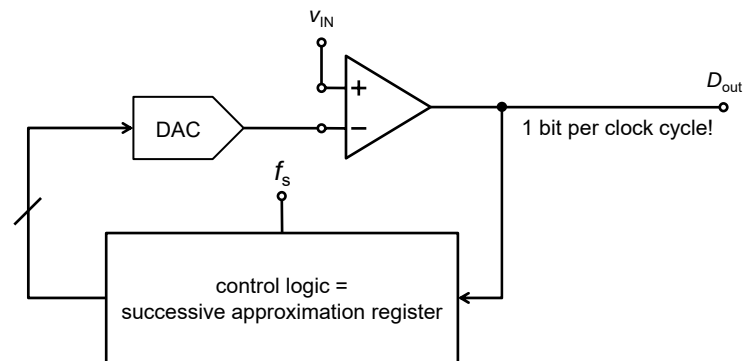
Data Converters

48

UNI  
FREIBURG



## Successive Approximation ADC



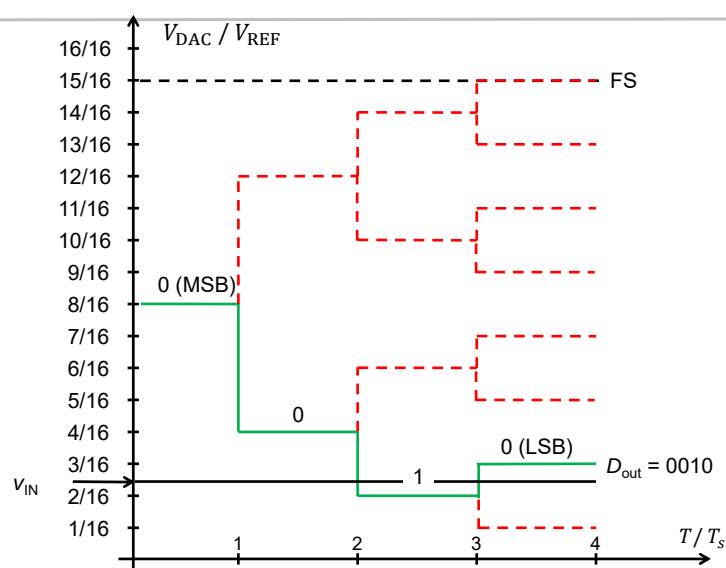
26-06-25

Data Converters

49

UNI  
FREIBURG

## Successive Approximation – Binary Search



26-06-25

Data Converters

50

UNI  
FREIBURG